Neural Architecture Optimization

Renqian Luo, Fei Tian, Tao Qin and Tie-Yan Liu

Presented by Niu Shuaicheng
Scut Artificial Intelligence Laboratory

November 4, 2018

Outline

1. Background
2. Proposed Methods
3. Experimental Results
4. Conclusion

Outline

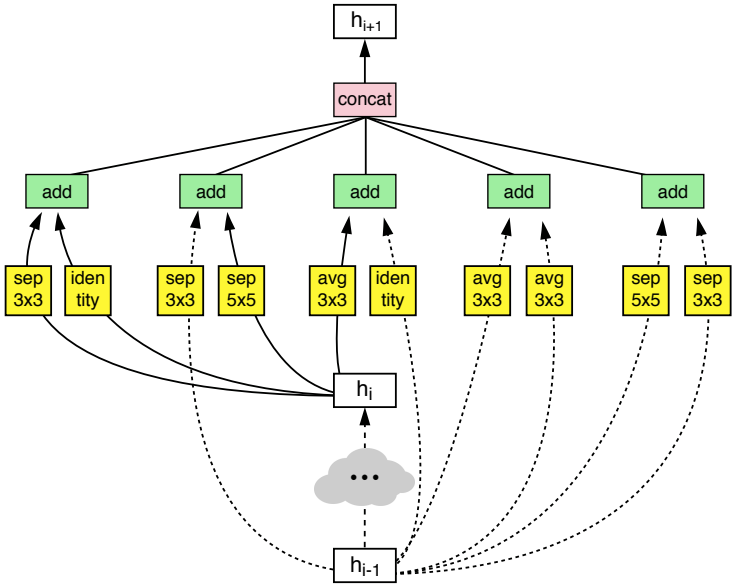
1. Background
2. Proposed Methods
3. Experimental Results
4. Conclusion

Related Work

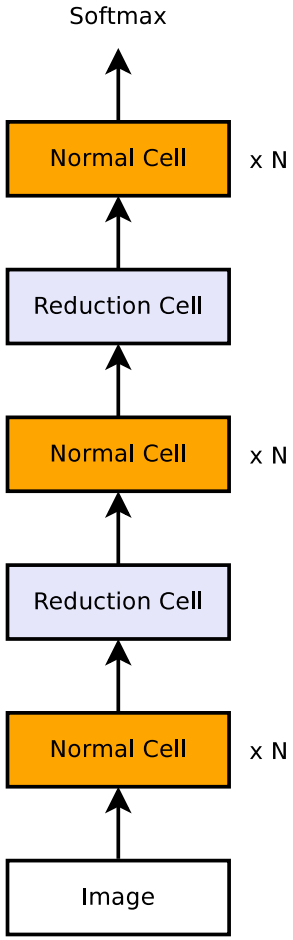
Categorized by search methods:

- Reinforcement learning (RL)
- Evolutionary algorithms (EA)
- Sequential model-based optimization (SMBO)

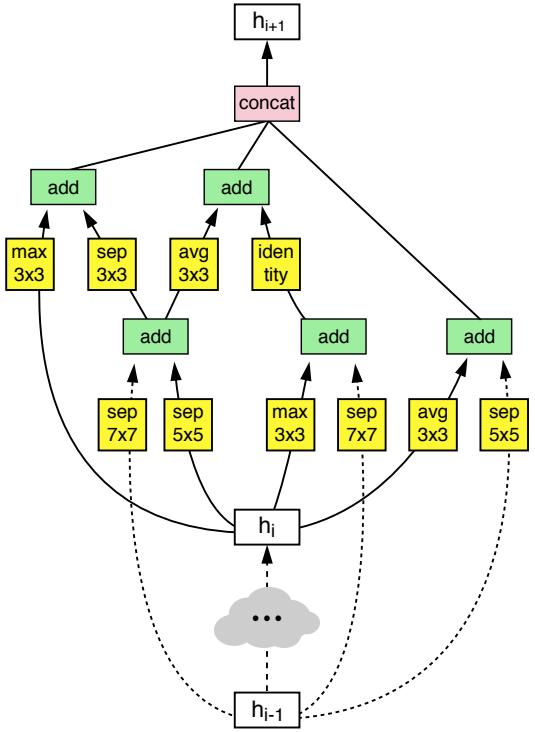
NASNet-RL Based



Normal Cell

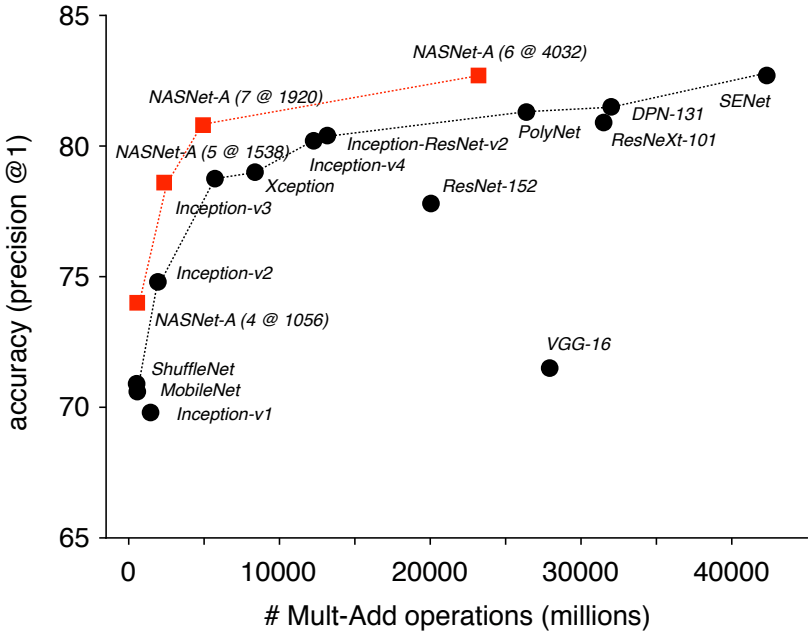


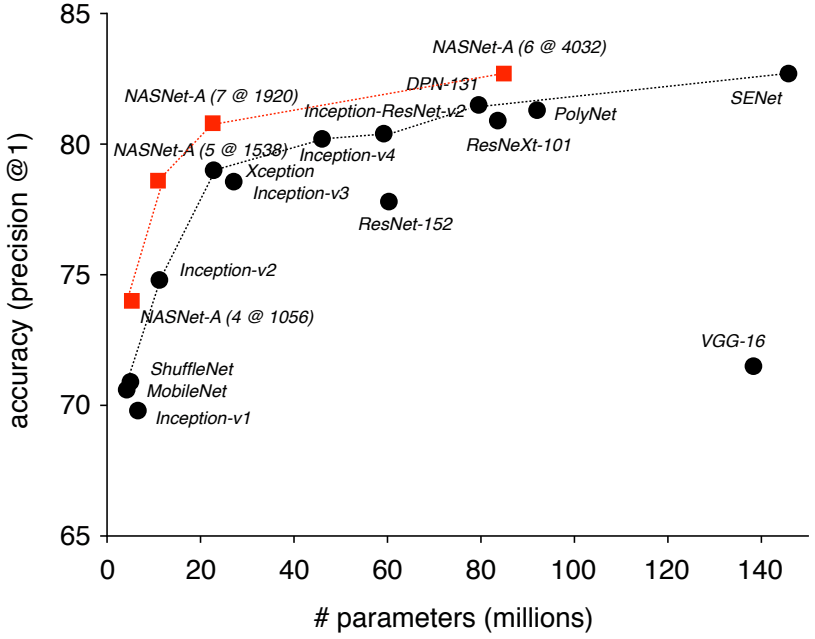
CIFAR10
Architecture



Reduction Cell

Performance on ImageNet





Efficiency of Architecture Search Methods

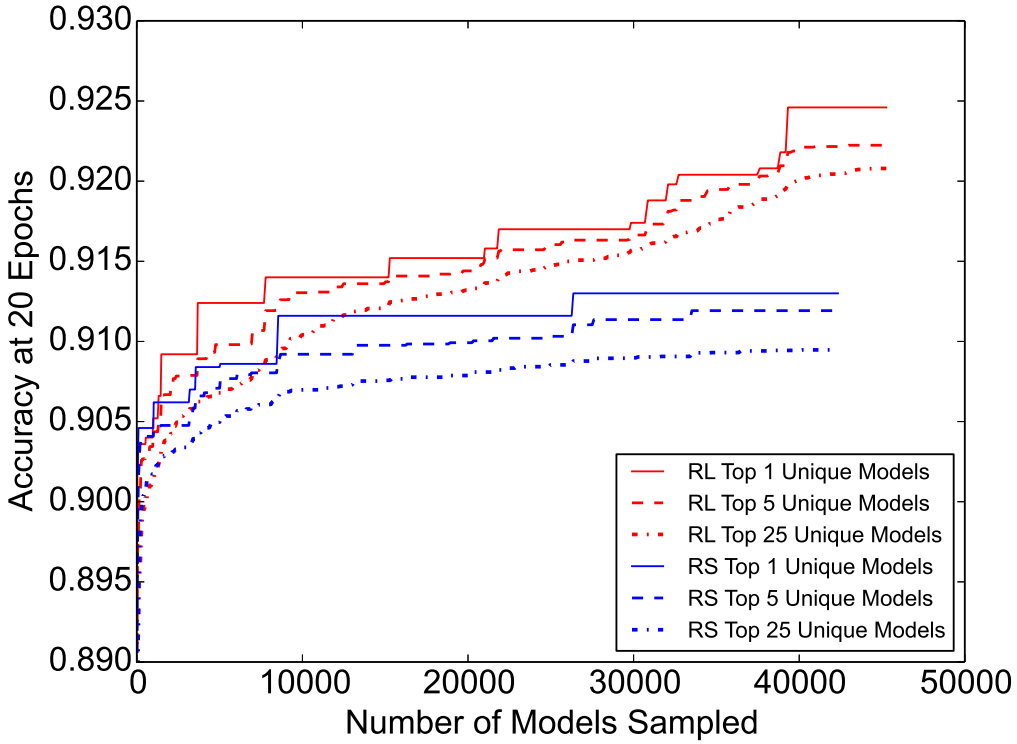


Figure. The efficiency of random search (RS) and reinforcement learning (RL) for learning neural architectures. The x-axis measures the total number of model architectures sampled, and the y-axis is the validation performance on CIFAR-10 after 20 epochs of training.

Computation Consuming of NASNet

500 GPUs $\times$ 4 days !!!

Motivation

Perform gradient-based optimization to accelerate the search process.

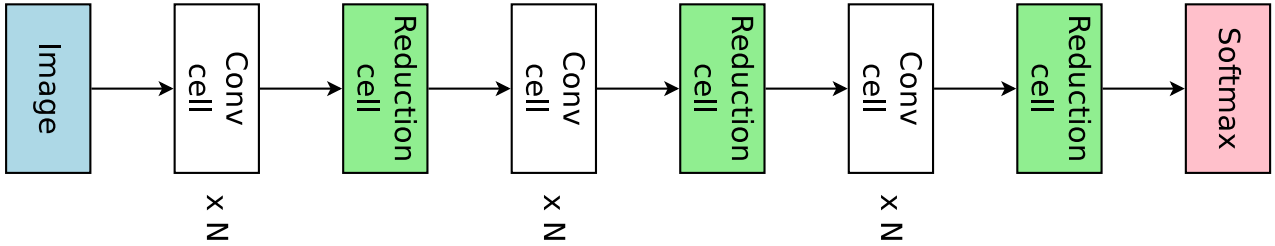
- Using a predictor to predict the performance of architectures' continuous representation
- Gradient ascent can be directly applied to the predictor to get better architectures
- Different from RL-based methods, there is no trial and error so that all new models sampled are efficient

Outline

1. Background
2. Proposed Methods
3. Experimental Results
4. Conclusion

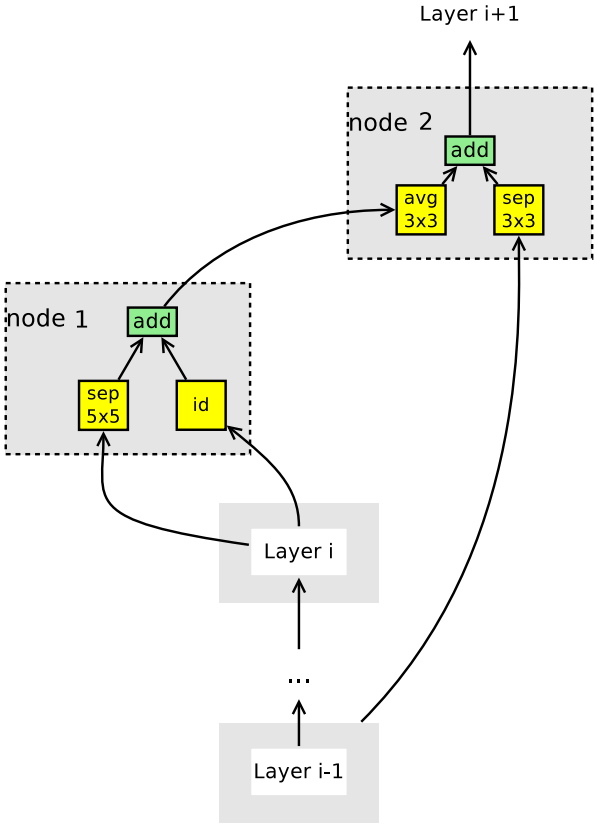
Designing Convolutional Networks

- Design a cell structure then stack them to form a final CNN architecture.
- This paper adopts the same cell architecture space commonly used in previous works [NAS, NASNet, ENAS, PNAS, AmoebaNet].



Cell Topologies

The cell can be represented by a DAG consisting of B nodes. Each node is a mapping from 2 input tensors to 1 output tensor and can be specified as a 4-tuple.



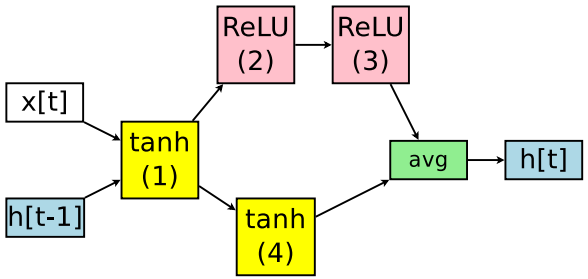
Definition

Node tuple in CNN cell: $< I_1, I_2, O_1, O_2 >$

- $I_1, I_2 \in \mathcal{I}_b$ specifies the inputs to the node
- $O_1, O_2 \in \mathcal{O}$ specifies the operation to apply to input I_i
- $\mathcal{I}_b = \{H_1^c, ..., H_{b-1}^c, H_B^{c-1}, H_B^{c-2}\}$ is the set of outputs of all previous nodes in this cell and the outputs of two previous cell
- $\mathcal{O}$ is the set of 11 functions, each of which operates on a single tensor

Designing Recurrent Networks

- Design a cell structure then stack them to form a final RNN architecture.
- The cell is consisting of B nodes. Each node can be specified as a 2-tuple.

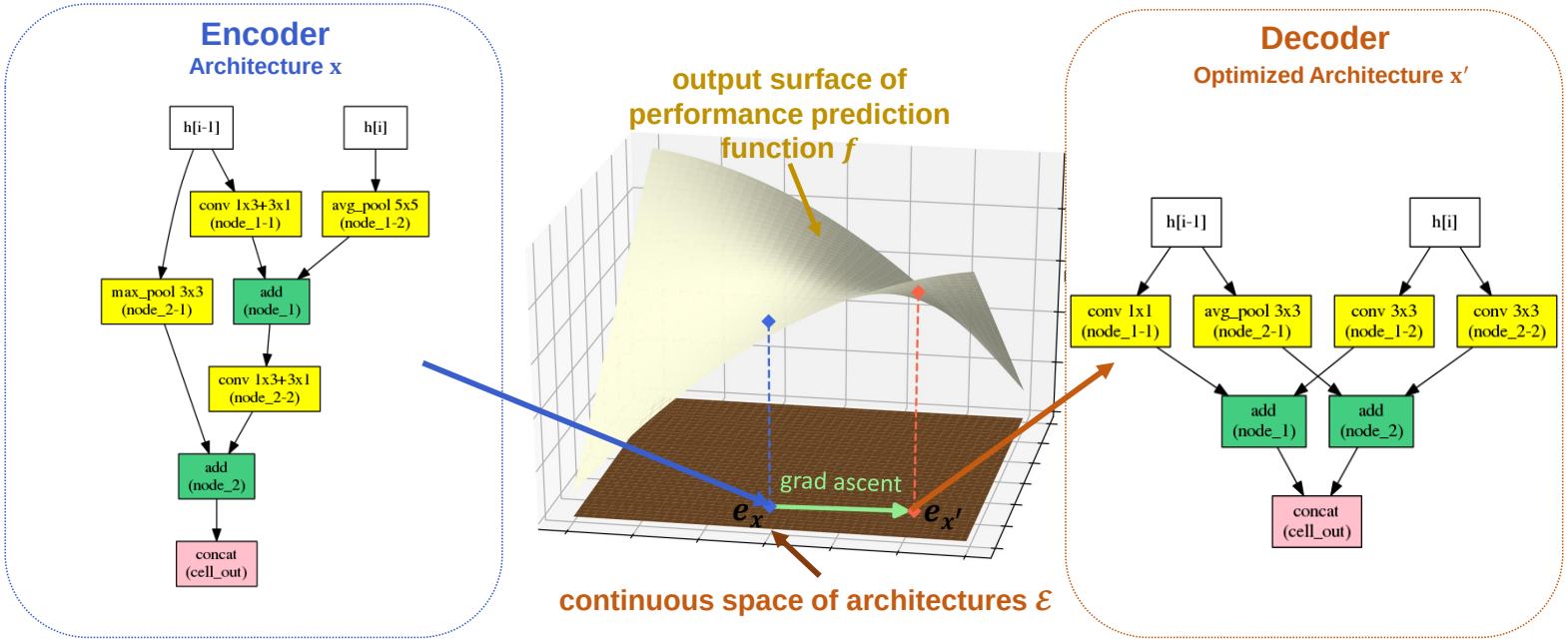


Definition

Node tuple in RNN cell: $< I, O >$

- $I \in \mathcal{I}_b$ is the input of this node
- $O \in \mathcal{O}$ is the operation to apply to input I
- $\mathcal{I}_b = \{H_1^c, ..., H_{b-1}^c\}$ is the set of outputs of all previous nodes in this cell
- $\mathcal{O}$ is the set of 4 operators

The Overall Framework of NAO



- The original architecture x is mapped to continuous representation e_x via encoder network
- e_x is optimized into $e_{x'}$ via maximizing the output of performance predictor f using gradient ascent
- $e_{x'}$ is transformed into a new architecture x' using the decoder network

Component of NAO: Encoder

Encoder $E : \mathcal{X} \to \xi$

E maps the string sequence describing an architecture into a continuous space:

- Input: an architecture $x \in \mathcal{X}$ (discrete space)
- Model: LSTM
- Output: $e_x = \{h_1, h_2, ..., h_T\} \in \xi = \mathcal{R}^{T \times d}$, where $h_t \in \mathcal{R}^d$ is LSTM hidden state at t-th timestep with dimension d (continuous space)

Component of NAO: Performance Predictor

Performance Predictor $f : \xi \rightarrow \mathcal{R}^+$

f maps the continuous representation e_x of an architecture x into its performance, i.e., accuracy:

- Input: $\overline{e}_x = \frac{1}{T}\sum_t^T h_t$ is the mean pooling result of e_x
- Model: a feed-forward network
- Output: the accuracy of architecture x
- Loss: the least-square regression loss $(s_x - f(E(x)))^2$, where s_x is the real accuracy on dev set

Component of NAO: Decoder

Decoder $D : \xi \to \mathcal{X}$

D decodes the string tokens x from its continuous representation e_x:

- Input: $e_x = \{h_1, h_2, ..., h_T\}$ (continuous space)
- Model: LSTM
- Output: architecture x

Objective:

$$\log P_D(x|E(x)) = \sum_{r=1}^{T} \log P_D(x_r|E(x), x_{<r}),$$

$$P_D(x_r|E(x), x_{<r}) = \frac{\exp(W_{x_r}[s_r, ctx_r])}{\sum_{x' \in V_r} \exp(W_{x'}[s_r, ctx_r])}$$

where W is the output embedding matrix for all tokens, $x_{<r}$ represents all the previous tokens before position r, s_r is the LSTM hidden state at r-th timestep, $[,]$ means concatenation of two vectors, and V_r denotes the space of valid tokens at position r.

Training

This paper jointly trains the encoder E, performance predictor f and decoder D by minimizing the combination of performance prediction loss L_{pd} and structure reconstruction loss L_{rec}:

$$L = \lambda L_{pd} + (1-\lambda) L_{rec} = \lambda \sum_{x \in X} (s_x - f(E(x)))^2 - (1-\lambda) \sum_{x \in X} \log P_D(x|E(x))$$

- X: all candidate architectures x
- λ: the trade-off parameter

Inference

This paper obtains a better continuous representation $e_{x'}$ by moving $e_x = \{h_1, h_2, ..., h_T\}$ towards the gradient direction induced by f:

$$h'_t = h_t + \eta \frac{\partial f}{\partial h_t}, \;\; e_{x'} = \{h'_1, h'_2, ..., h'_T\}$$

Detailed Algorithm

Algorithm 1 Neural Architecture Optimization

Input: Initial candidate architectures set X to train NAO model. Initial architectures set to be evaluated denoted as $X_{eval} = X$. Performances of architectures $S = \emptyset$. Number of seed architectures K. Step size η. Number of optimization iterations L.

for $l = 1, \cdots, L$ **do**

Train each architecture $x \in X_{eval}$ and evaluate it to obtain the dev set performances $S_{eval} = \{s_x\}, \forall x \in X_{eval}$. Enlarge S: $S = S \bigcup S_{eval}$.

Train encoder E, performance predictor f and decoder D by minimizing Eqn.(1), using X and S.

Pick K architectures with top K performances among X, forming the set of seed architectures X_{seed}.

For $x \in X_{seed}$, obtain a better representation $e_{x'}$ from $e_{x'}$ using Eqn. (2), based on encoder E and performance predictor f. Denote the set of enhanced representations as $E' = \{e_{x'}\}$.

Decode each x' from $e_{x'}$ using decoder, set X_{eval} as the set of new architectures decoded out: $X_{eval} = \{D(e_{x'}), \forall e_{x'} \in E'\}$. Enlarge X as $X = X \bigcup X_{eval}$.

end for

Output: The architecture within X with the best performance

Outline

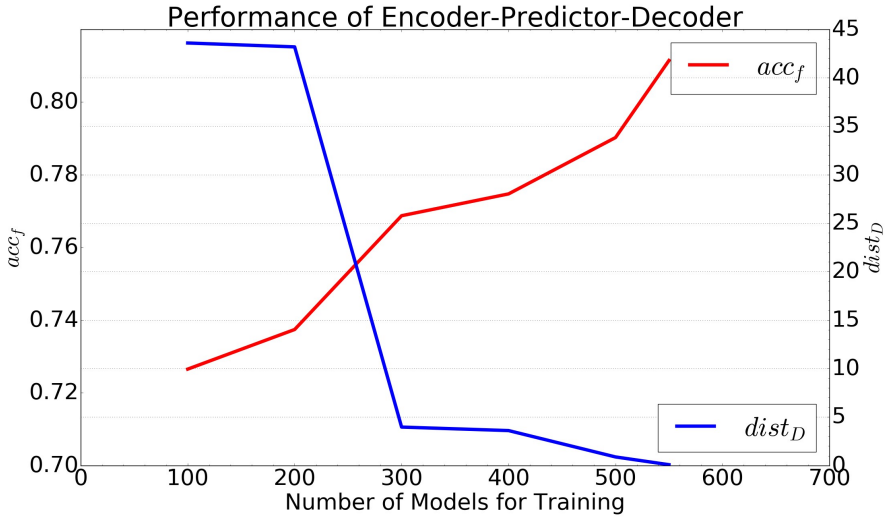
1. Background
2. Proposed Methods
3. Experimental Results
4. Conclusion

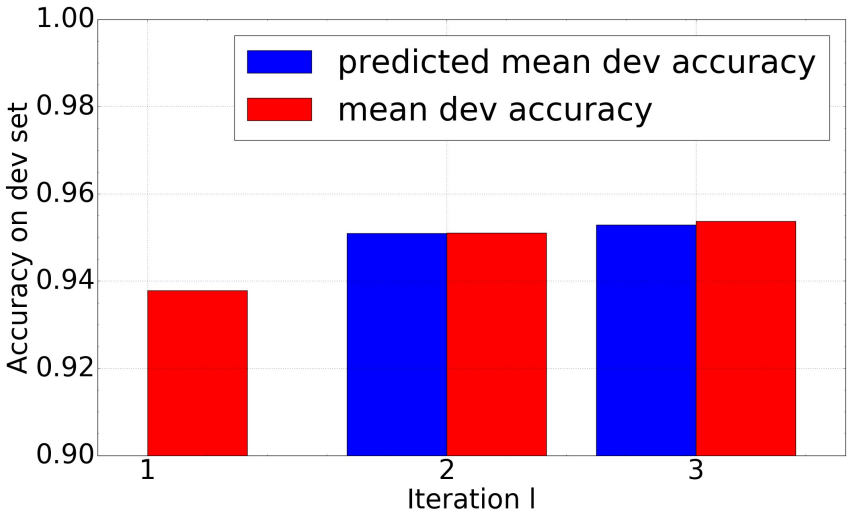
Results on CIFAR-10 Classification

Model	B	N	F	#op	Error(%)	#params	M	GPU Days
DenseNet-BC [19]		100	40	/	3.46	25.6M	/	/
ResNeXt-29 [43]				/	3.58	68.1M	/	/
NASNet-A [47]	5	6	32	13	3.41	3.3M	20000	2000
NASNet-B [47]	5	4	N/A	13	3.73	2.6M	20000	2000
NASNet-C [47]	5	4	N/A	13	3.59	3.1M	20000	2000
Hier-EA [27]	5	2	64	6	3.75	15.7M	7000	300
AmoebaNet-A [38]	5	6	36	10	3.34	3.2M	20000	3150
AmoebaNet-B [38]	5	6	36	19	3.37	2.8M	27000	3150
AmoebaNet-B [38]	5	6	80	19	3.04	13.7M	27000	3150
AmoebaNet-B [38]	5	6	128	19	2.98	34.9M	27000	3150
AmoebaNet-B + Cutout [38]	5	6	128	19	2.13	34.9M	27000	3150
PNAS [26]	5	3	48	8	3.41	3.2M	1280	225
ENAS [36]	5	5	36	5	3.54	4.6M	/	0.45
Random-WS	5	5	36	5	3.92	3.9M	/	0.25
DARTS + Cutout [28]	5	6	36	7	2.83	4.6M	/	4
NAONet	5	6	36	11	3.18	10.6M	1000	200
NAONet	5	6	64	11	2.98	28.6M	1000	200
NAONet + Cutout	5	6	128	11	2.11	128M	1000	200
NAONet-WS	5	5	36	5	3.53	2.5M	/	0.3

- B: the number of nodes within a cell
- N: the number of the times the discovered normal cell is unrolled to form final CNN
- F: the filter size
- #op: the number of different operation for one branch in the cell
- M: the total number of architectures that are trained

Efficiency of Encoder-Predictor-Decoder



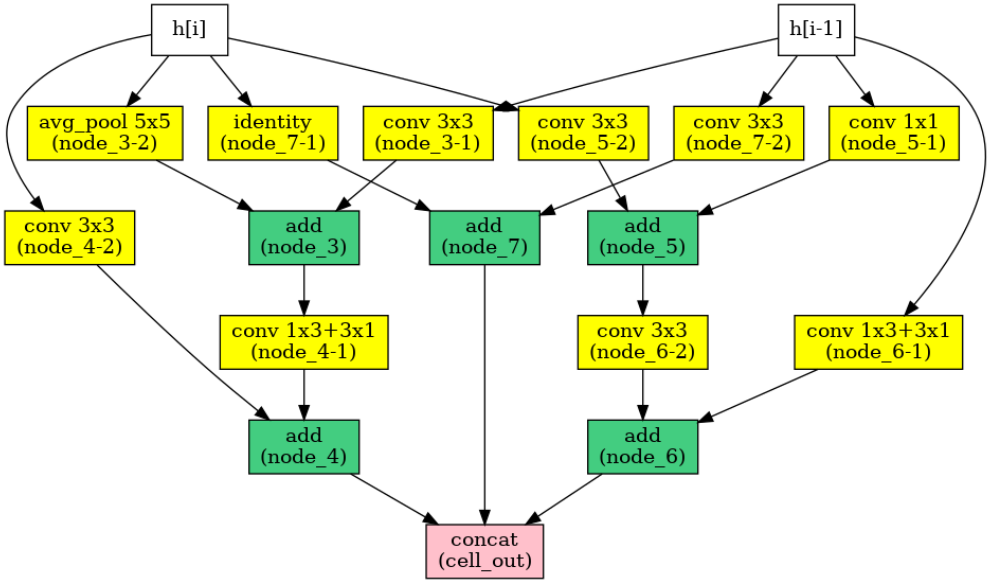


- acc_f: the accuracy (not error rate) of predictor f,
  $acc_f = \frac{\sum_{x_1 \in X_{test}, x_2 \in X_{test}} \mathbb{I}_{f(E(x_1)) \geq f(E(x_2))} \mathbb{I}_{s_{x_1} \geq s_{x_2}}}{\sum_{x_1 \in X_{test}, x_2 \in X_{test}}}$
- $dist_D$: the average distance of $D(E(x))$ and x, $dist_D = \frac{1}{|X_{test}|} \sum_{x \in X_{test}} Dist(D(E(x), x))$, where $Dist$ is Hamming distance
- mean dev accuracy: $\frac{1}{|X_{eval}|} \sum_{x \in X_{eval}} s_x$
- predicted mean dev accuracy: $\frac{1}{|X_{eval}|} \sum_{x \in X_{eval}} f(E(x))$

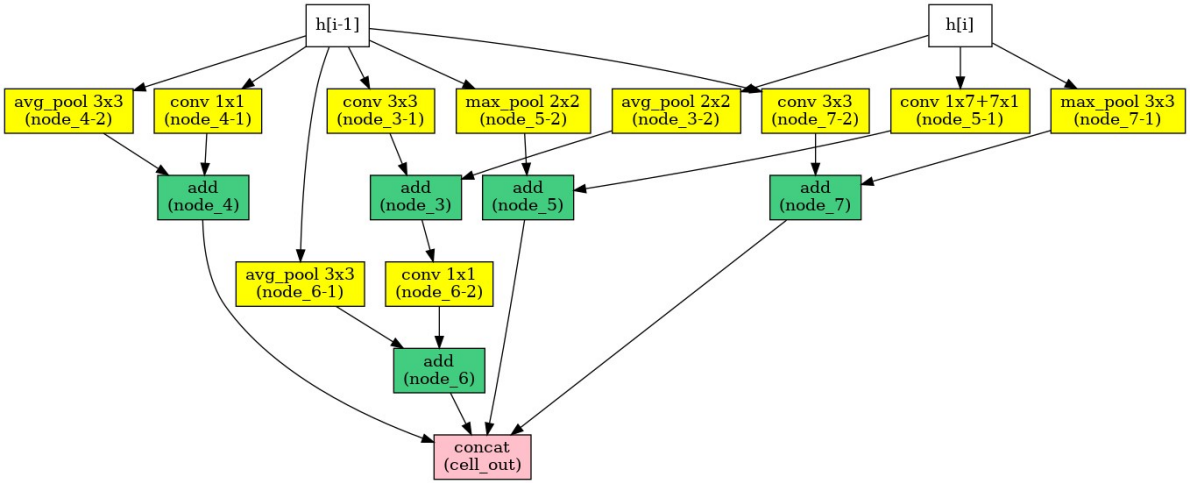
Results of Language Modeling on PTB

Models and Techniques	#params	Test Perplexity	GPU Days
Vanilla LSTM [45]	66M	78.4	/
LSTM + Zoneout [23]	66M	77.4	/
Variational LSTM [14]	19M	73.4	
Pointer Sentinel-LSTM [33]	51M	70.9	/
Variational LSTM + weight tying [20]	51M	68.5	/
Variational Recurrent Highway Network + weight tying [46]	23M	65.4	/
4-layer LSTM + skip connection + averaged weight drop + weight penalty + weight tying [31]	24M	58.3	/
LSTM + averaged weight drop + Mixture of Softmax + weight penalty + weight tying [44]	22M	56.0	/
NAS + weight tying [47]	54M	62.4	1e4 CPU days
ENAS + weight tying + weight penalty [36]	24M	58.6[5]	0.5
Random-WS + weight tying + weight penalty	27M	58.81	0.4
DARTS+ weight tying + weight penalty [28]	23M	56.1	1
NAONet + weight tying + weight penalty	27M	56.0	300
NAONet-WS + weight tying + weight penalty	27M	56.6	0.4

Best Architecture Cell of CNN

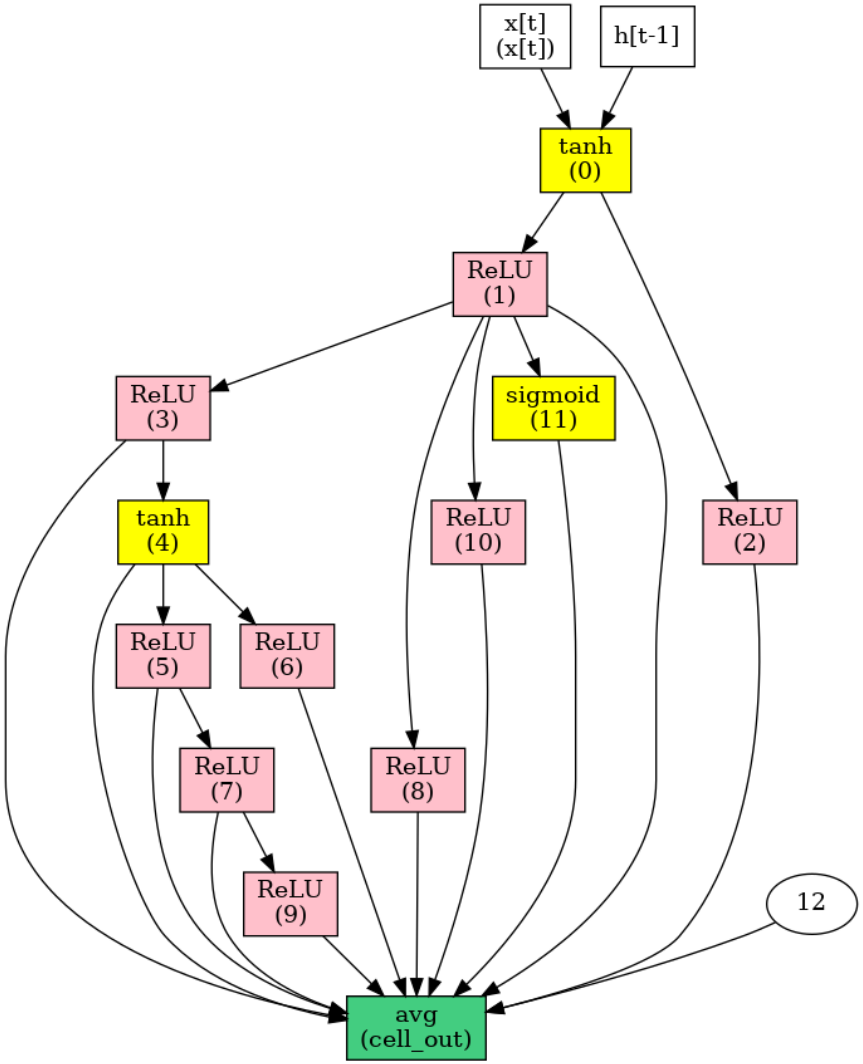


(a) Normal Cell



(b) Reduction Cell

Best Architecture Cell of RNN



Outline

1. Background
2. Proposed Methods
3. Experimental Results
4. Conclusion

Conclusion

Conclusion

- This paper proposes a new idea of designing architecture automatically within continuous space rather than searching discrete decisions, which speed up the searching convergence a lot
- The transfer ability of this algorithm on big datasets such as ImageNet is still potential to improve

Thank you for your attention!